TECHNICAL PROJECT REPORT — SOFTWARE ENGINEERING

Multi-Tenant URL Shortener & Analytics Platform

A full-stack SaaS platform for tenant-isolated link management, click analytics, and role-based access control

Stack	**NestJS · React · PostgreSQL · Redis · Prisma · TypeScript**
Architecture	**REST API monorepo (npm Workspaces)**
Deployment	**Docker Compose (PostgreSQL 16 + Redis 7)**

1. Abstract

This project delivers a **production-oriented** multi-tenant URL shortening and analytics platform. Organisations (tenants) are provisioned as isolated namespaces on shared infrastructure, each managing their own short links, users, and click telemetry.

The system exposes two distinct traffic paths: an authenticated management API (link CRUD, analytics dashboards, user management) protected by a three-stage NestJS guard chain, and a public redirect path optimised for minimal latency via a Redis cache-aside strategy. Redis serves cached redirects without requiring a PostgreSQL lookup, and click events are recorded asynchronously in PostgreSQL, keeping the redirect response on the fast path.

Role-based access control (RBAC) distinguishes three roles — **MEMBER**, **TENANT_ADMIN**, **SUPER_ADMIN** — enforced through a permission-based guard and a Prisma Client Extension that transparently injects tenant scoping into every database query. The React frontend presents tenant-scoped dashboards and a platform-wide admin console. The goal is correct data isolation, high redirect throughput, and an extensible architecture ready for horizontal scale.

2. Problem Statement

SaaS teams building URL shorteners face two competing concerns: shared infrastructure cost (multiple tenants on one deployment) and strict data isolation (tenant A must never see tenant B's links or analytics). Existing open-source shorteners are single-tenant by design; retrofitting isolation via application-level filters is error-prone because a single missed `WHERE tenantId = ?` clause can cause a cross-tenant data leak.

Additionally, the redirect endpoint is the highest-traffic route and must not be gated behind authentication overhead, yet click attribution must still be correctly stamped per tenant. The platform is designed to resolve both concerns at the architecture level rather than relying on developer discipline alone.

3. Objectives

- **Tenant isolation by construction:** centralise tenant filtering in a Prisma Client Extension (`TenantPrismaService`) so that accidental unscoped queries against tenant-owned tables are prevented, rather than relying on every call site remembering to filter.
- **Latency-optimised redirects:** serve short-link redirects from Redis using a cache-aside strategy, with TTL bounded by the link's `expiresAt`. Redis serves cached redirects without requiring a PostgreSQL lookup; click events are recorded asynchronously in PostgreSQL after the redirect response has already been sent.
- **Granular RBAC:** implement a permission-based guard covering URL creation, management, analytics, and platform-level admin operations, driven by a centralised permission-to-role mapping table.
- **Secure token lifecycle:** issue short-lived JWT access tokens (15 min) and 7-day rotating refresh tokens; hash refresh tokens with SHA-256 and compare using `timingSafeEqual`.
- **Observability foundation:** structured JSON logging (`nestjs-pino`), Redis-backed rate limiting, and a dedicated health endpoint to support future production deployment.

4. Existing vs. Proposed System

⚠ Existing Limitations

- Single-tenant URL shorteners (e.g. YOURLS, Polr) with no isolation model
- Application-level filtering — every developer must remember `WHERE tenantId` at every call site
- Redirect and management traffic share the same pipeline, adding unnecessary auth overhead
- No standardised RBAC; permissions hardcoded per endpoint or absent entirely

✓ Proposed System

- Tenant scoping enforced by a Prisma Client Extension via `TenantPrismaService` — centralised scoping prevents accidental unscoped queries
- Redirect path registered on raw Express, before the NestJS router — bypasses all guard overhead entirely
- Declarative permissions (`@RequirePermissions()`) with a centralised permission-to-role mapping table
- Rotating refresh tokens with a hash stored per user row; logout invalidates the session immediately

> **Refresh-token design, exactly as implemented:** the refresh-token hash is stored as a single column on the user row. Each successful login overwrites that hash, so there is one active refresh token per user at a time. This gives secure rotation and immediate invalidation on logout for a single active session — it does not implement multi-device session management (tracking several concurrently valid refresh tokens per user), since that is not what the current schema stores.

Why this matters for the redirect path

Because the redirect handler is registered on the raw Express layer *before* the NestJS router is reached, a redirect request never enters the JWT/RBAC/tenant-context guard chain at all — it is not merely "fast," it is structurally a different, shorter code path than every authenticated management request.

5. System Architecture

FIG. 1 — SYSTEM ARCHITECTURE: TWO DISTINCT TRAFFIC PATHS ON SHARED INFRASTRUCTURE

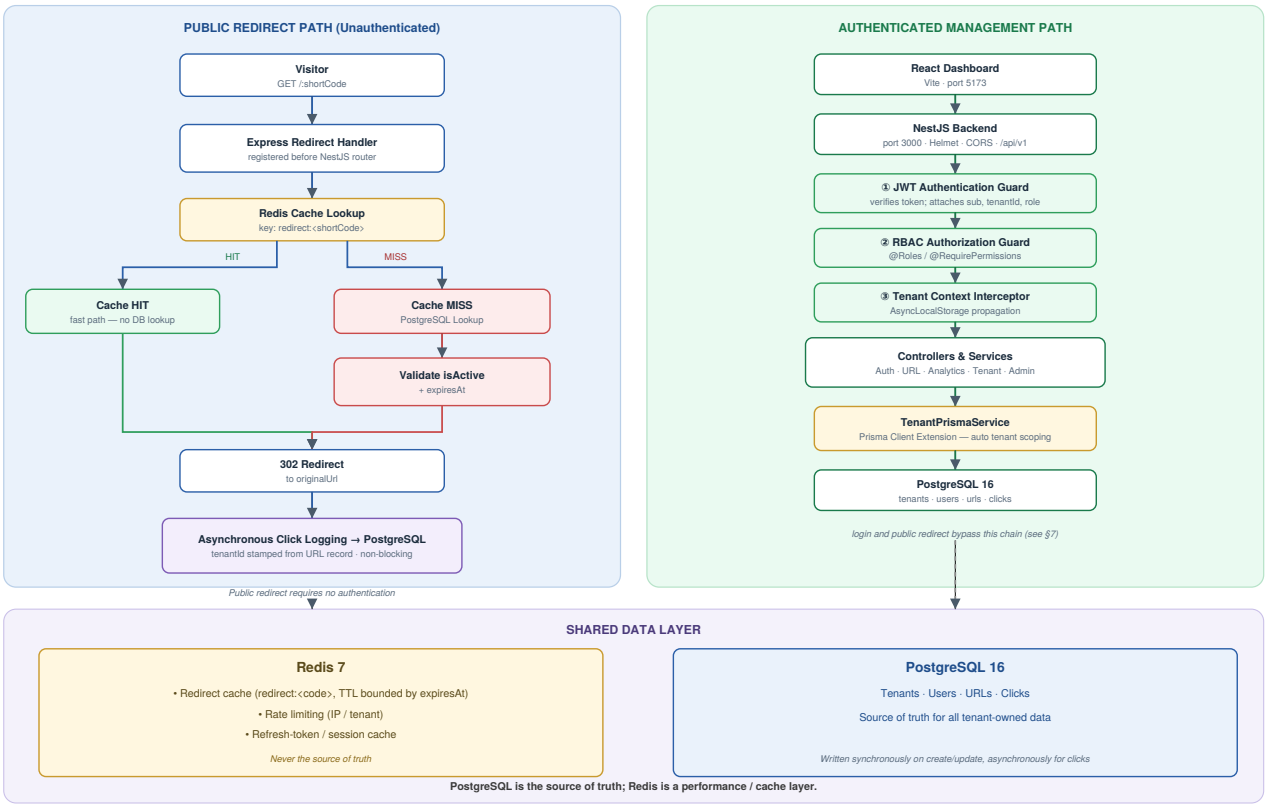

Fig. 1 — Two distinct traffic paths (public redirect vs. authenticated management) on shared PostgreSQL/Redis infrastructure.

Redis / PostgreSQL relationship

Redis serves cached redirects without requiring a PostgreSQL lookup — a cache hit resolves entirely in Redis. On a cache miss, PostgreSQL is consulted, the result is validated (`isActive`, `expiresAt`) and written back into Redis with a bounded TTL. Click events are always recorded asynchronously in PostgreSQL, after the 302 response has already been sent, so they never add latency to the redirect itself.

> PostgreSQL is the source of truth for all tenant-owned data; Redis is a performance/cache layer and is never treated as authoritative.

Tenant isolation & security mechanisms

`TenantPrismaService` wraps Prisma via a Client Extension, merging `tenantId` into every `where`/`data` clause at query time. `AsyncLocalStorage` propagates the tenant ID from the JWT through the entire async call chain with no manual argument threading. The extension fails closed: if no tenant context is active, it throws a `ForbiddenException` rather than executing an unscoped query.

> **Exceptions to tenant-context enforcement:** `login` and the public `redirect` handler deliberately use the raw `PrismaService` instead of `TenantPrismaService`, because both operate before any tenant identity is known — they look records up by a globally unique identifier (email for login, `shortCode` for redirect) rather than by a caller's tenant membership. Tenant scoping only becomes meaningful once a caller identity exists.

6. Methodology & Technical Pipeline

FIG. 2 — TECHNICAL PIPELINE: URL CREATION FLOW AND REDIRECT FLOW

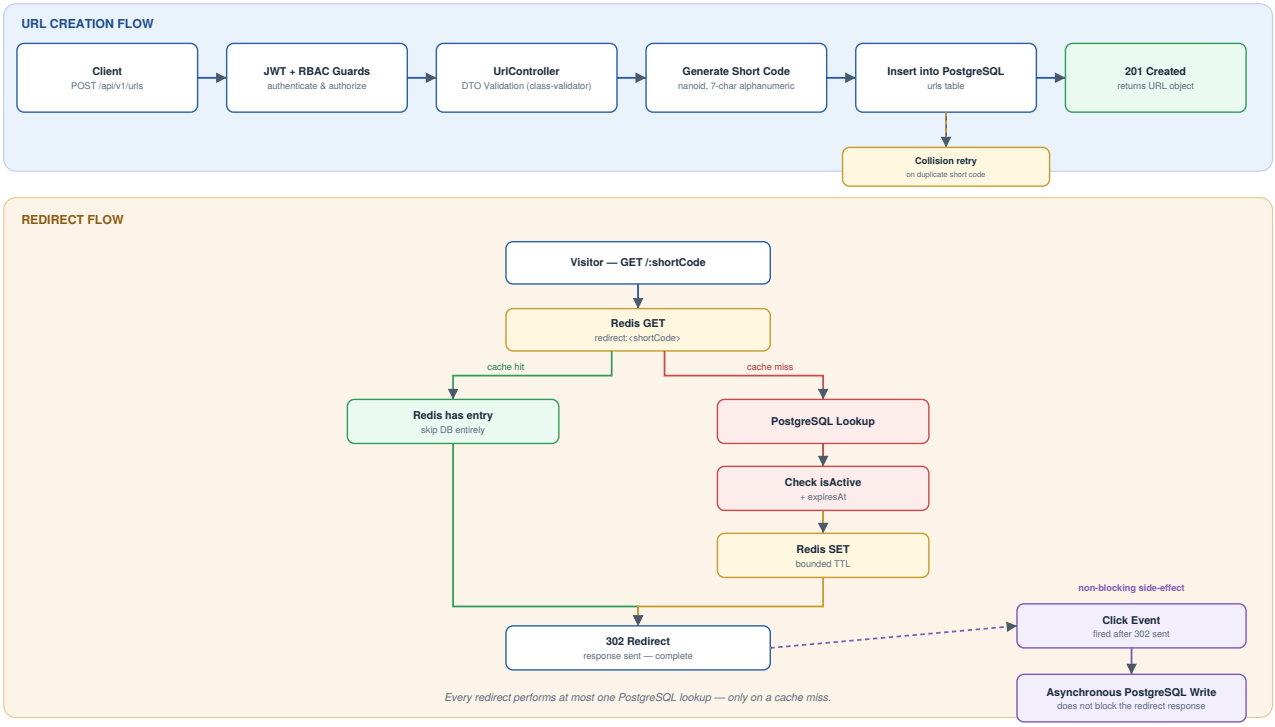

Fig. 2 — URL creation flow and redirect flow, shown separately. Every redirect performs at most one PostgreSQL lookup (only on a cache miss); the click write is asynchronous and never blocks the response.

7. Backend Module Architecture & RBAC Design

Module	Responsibilities
AuthModule	POST /auth/login · /auth/refresh · /auth/logout · /auth/register — bcrypt cost 12, SHA-256 refresh-token hash
UrlModule	POST/GET/PATCH/DELETE /urls (soft delete, paginated listing) — nanoid short codes
Analytics	GET /analytics/overview, /clicks, /referrers, /user-agents — tenant-scoped queries, browser/OS parsing
RedirectModule	Express handler registered before NestJS · GET /:shortCode · RedirectCacheService · RESERVED_PATHS bypass
AdminModule SUPER_ADMIN only	Platform-wide stats, tenant provisioning, user/link administration, cross-tenant analytics

RBAC permission matrix

Permission	MEMBER	TENANT_ADMIN	SUPER_ADMIN
url:read	✓	✓	✓
url:create	✓	✓	✓
url:manage_own	✓	✓	✓
url:manage (any)	—	✓	✓
analytics:read	✓	✓	✓
tenant:manage	—	✓	✓
platform:read / manage	—	—	✓

8. Technology Stack

Category	Technology / Version
Language	TypeScript 5.7 (strict mode, monorepo)
Backend Framework	NestJS 11 (Express adapter)
Frontend	React 18.3 + Vite 6 + React Router 7
Database	PostgreSQL 16 (Docker-managed)
ORM	Prisma 6.3 (Client Extension for tenant scoping)
Cache / Rate Limit	Redis 7 (ioredis 5.4)
Authentication	JWT (@nestjs/jwt); bcrypt 6; SHA-256 refresh hash
Short Code Generation	nanoid 3 (custom alphanumeric alphabet, 7 characters)
HTTP Security	Helmet 8; CORS; ValidationPipe (class-validator)
Logging	nestjs-pino (structured JSON); pino-pretty (dev)
Testing	Jest 29 + ts-jest (url.service.spec, analytics.service.spec)
Infrastructure	Docker Compose; npm Workspaces monorepo

9. Complete End-to-End Workflow

FIG. 3 — COMPLETE END-TO-END WORKFLOW

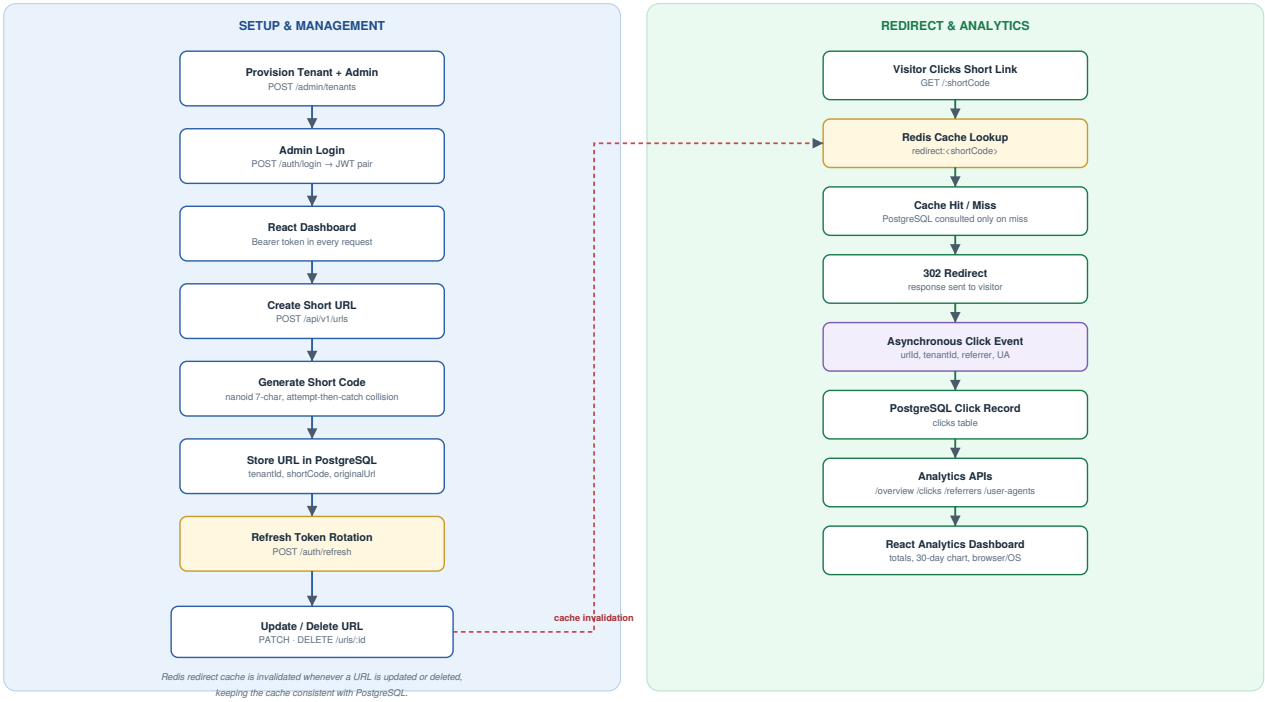

Fig. 3 — Setup/management flow (left) and redirect/analytics flow (right). Updating or deleting a URL invalidates its Redis redirect-cache entry, keeping the cache consistent with PostgreSQL.

FIG. 4 — DATABASE / ENTITY-RELATIONSHIP DIAGRAM

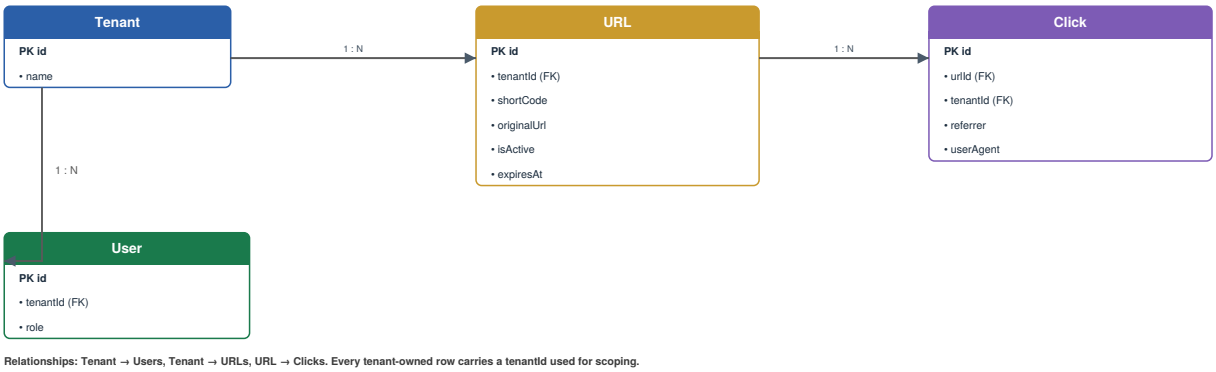

Fig. 4 — Core schema. Tenant → Users and Tenant → URLs are one-to-many; URL → Clicks is one-to-many. No fields beyond those implemented are shown.

10. Key Design Decisions & Verified Behaviours

Redirect Correctness — Verified

- Link created with a 3-second `expiresAt`
- Redis TTL matched `expiresAt`
- 302 returned before expiry
- 410 Gone returned immediately after expiry

Timing Attack Prevention — Verified

- Login runs a bcrypt comparison even for unknown emails, against a fixed decoy hash
- Removes the timing difference between "no such user" and "wrong password" responses

Tenant Isolation — Verified

- Two tenants seeded independently
- Cross-tenant lookup via `TenantPrismaService` for a valid ID belonging to another tenant returned `null`
- Tenant-owned rows remained inaccessible even with a correct ID

Security Verification Summary

- Fail-closed tenant scoping (throws rather than running unscoped)
- SHA-256 + `timingSafeEqual` refresh-token comparison
- Helmet, CORS, and DTO validation on every management route

11. Conclusion & Future Scope

The platform achieves its core goals: correct tenant isolation enforced at the ORM layer, a latency-optimised public redirect path, declarative RBAC, secure token rotation, and a React dashboard covering link management and per-tenant click analytics. The deliberate separation of the redirect and management code paths allows each to be optimised independently without compromising the other.

Deferred / Future Improvements

- **Async click recording:** queue-backed write (BullMQ) to decouple redirect latency from database write latency.
- **Immediate access-token revocation:** a Redis denylist to close the 15-minute post-logout window.
- **Analytics at scale:** a dedicated time-series store or event stream once click volume exceeds relational comfort.
- **Edge / Nginx layer:** TLS termination and edge-level rate limiting for production deployment.
- **Schema-per-tenant:** justified only when contractual data residency or dedicated SLAs are required.